

DisloCluster Code History

Every change from MoDELib2-NNL to the present, numbered,
beginning with the immobile population equations

DisloCluster development note

15 August 2026

Abstract

This document records, in one continuous numbering, every substantive change made on the way from MoDELib2-NNL to DisloCluster. Each entry states *what* was changed, *why*, what it cost or bought in *code* structure, and what it did to *computational efficiency*.

The baseline is the point at which the **immobile population equations were introduced** into MoDELib2-NNL — which until then had no immobile species at all — to make the 3-D code consistent with the ZrMicro 0-D model: loop nucleation, absorption growth, vacancy-loop annealing, loop-loop coalescence and loop-network coalescence. Those occupy Part [I](#) and are Changes [1–13](#). Everything after them follows from consequences that only became visible once the immobile families were alive: the mobile solve had to be repaired to feed them (Part [II](#)), the material file had to be made to agree with the 0-D calibration (Part [III](#)), a genuine field-level coupling had to replace a four-scalar handoff (Part [IV](#)), the conservation bookkeeping had to be reinterpreted for an open system (Part [V](#)), and the cost of the resulting scheme had to be brought down by a factor of several (Part [VI](#)).

Two conventions are used throughout. First, a change is reported with the number that was *measured*, not the number that was expected; where a change bought nothing, that is stated. Second, correctness checks that were available in principle but not exercised are not claimed as verification. The open items are collected, unresolved, in Part [VIII](#).

Contents

1	The baseline: MoDELib2-NNL, before the immobile populations	3
I	The immobile population equations	3
2	Introducing the immobile populations	4
3	Nucleation	5
4	Growth by absorption	6
5	Annealing and thermal emission	7
6	Coalescence	8
II	The mobile solve, and the coupling between the two	9
III	The material file and the verification case	13

IV	The coupled architecture	15
V	Physics corrections found by measurement	17
VI	Computational efficiency	20
7	The 0-D solver	20
8	The fast solve	21
VII	Portability, reproducibility and post-processing	23
VIII	Open items	25
	Verification status against the 0-D	27
	Summary of measured efficiency changes	28

1 The baseline: MoDELib2-NNL, before the immobile populations

The baseline for this document is `mlm335/MoDELib2-NNL`, the state of the code at the moment the immobile population equations were introduced into it. That is where the work recorded here begins, and Change 1 is that introduction.

At that point the code solved the mobile steady-state cluster-dynamics PDE and *nothing else*. Its immobile species count `iSize` was `0`, `solveImmobileClusters()` was an empty function body, there was no `ImmobileSinkRate.h` and no immobile integrator of any kind, and the source carried a literal `// Missing immobile sinks placeholder` at `ClusterDynamicsFEM.cpp:110` where the loop absorption term belongs. The loop populations were not present in a simplified form — they were **absent**. Loops appeared in the model only as a hole where a sink term should be.

The immobile equations below were therefore written into that code to make it consistent with the **ZrMicro 0-D model**: cascade nucleation, absorption growth, vacancy-loop annealing, and coalescence in the loop-loop and loop-network channels, each given the same meaning it carries in ZrMicro so that the 3-D solve reduces to the 0-D result in the well-mixed limit. The governing equations are those of Po & Ghoniem, Deliverable D1/M1, §2.2; equation numbers below refer to that document.

Above the baseline. This checkout records that the tree descends from **Po’s MoDELib2** — `README.md` reads `# MoDELib2` and `Contributors.txt` is headed by Giacomo Po. The registered submodule URL was `https://github.com/Ghoniem/MoDELib2-NNL.git`. Neither upstream is vendored here, so nothing further about the ancestry can be verified from the repository itself.

A note on `mlm335/MoDELib-fullCD`, and on an earlier error in these documents. `MoDELib-fullCD` is a *separate* cluster-dynamics branch: `iSize` is 8, and it carries `ImmobileSinkRate.h`, `SpatialODESolver.h`, `FirstOrderReaction.h` and a continuum $\rightarrow$ discrete loop conversion. Comparing the two repositories after the fact shows 31 cluster-dynamics parameter keys shared verbatim and a set of systematic renames (`loopNucDefects` $\rightarrow$ `nNuc`, `loopCoalLL/LN` $\rightarrow$ `cLL/cLN`, `loopCoalKappa*` $\rightarrow$ `kappa*`, `loopAnnealTau0_SI` $\rightarrow$ `tau0_vLoop_SI`, `loopCoalNetwork_SI` $\rightarrow$ `rhoNetwork_SI`, `minimumLoopSize` $\rightarrow$ `r_min`).

An earlier revision of these notes read that overlap as *descent* and declared `fullCD` the correct baseline, describing the immobile solver below as a re-discretization of `fullCD`’s. **That inference was wrong.** Shared naming across two branches of one code, developed in one group from one formulation, is evidence of *convergence*, not of derivation — and it says nothing about which came first. The immobile equations here were written to reproduce ZrMicro, not to re-express `fullCD`.

`fullCD` remains worth comparing against, and this document does so wherever the comparison is informative — most usefully in Change 2, where its Galerkin/mass-matrix discretization is set beside the nodal one adopted here, and in Part VIII, where its continuum $\rightarrow$ discrete loop conversion is noted as a capability this branch does not have. Those are comparisons between siblings, not a record of what was inherited.

Chronology. Changes 1–24 were made on the `zr3d_ghoniem` branch of the fork before the repository was consolidated. Changes 25 onward are dated by the repository’s own history, from `c7574f6` (7 August 2026) to `6dbcc68` (15 August 2026).

Part I

The immobile population equations

This is where the work started, and it started from nothing. `MoDELib2-NNL` had no immobile species: the loop populations that the ZrMicro 0-D model is calibrated on had no

representation in the 3-D code at all. Every kinetic channel in this part — cascade nucleation, absorption growth, vacancy-loop annealing, loop-loop coalescence, loop-network coalescence — was written into the code here, given the same meaning it carries in ZrMicro, and then integrated by a scheme that survives the stiffness those channels bring with them.

The changes divide into two kinds, and it is worth separating them because they failed in completely different ways. Changes 1–11 *introduce* physics: a channel that was not there before is added and given its 0-D form. Changes 6, 8, 9, 10 and 12 then *repair how those channels were integrated* — a term written in a form that could not enter an implicit update, a unit conversion missing so a gate never closed, or a channel integrated explicitly when its own time constant sat orders of magnitude below the step.

The second kind produced by far the larger errors. A missing channel is usually visible as a trend that is simply wrong; a channel present but integrated explicitly past its stability limit produces a population that resets itself every step, which looks like physics until the numbers are checked against an analytic balance. The loop content was wrong by a factor of 350 for that reason, not because the annealing term was absent.

2 Introducing the immobile populations

Change 1. `iSize 0 → 8`: the immobile families are created

The immobile degrees of freedom are introduced as four families $\times$ two quantities each — number density and defect content: $[N_c, N_{a1}, N_{a2}, N_{a3} \mid c_c, c_{a1}, c_{a2}, c_{a3}]$. One vacancy-type $\langle c \rangle$ family and three prismatic interstitial-type $\langle a \rangle$ families.

Why. The 0-D model carries exactly these populations, and the point of the exercise is that the 3-D reduce to it in the well-mixed limit. This is the change that brings the immobile half of the model into existence: before it, `iSize` was 0 and there was no loop population to evolve, no sink for the mobile species to see, and nothing for the ZrMicro parameter set to attach to.

Code. `ClusterDynamicsParameters.h`. The change is global — because `iSize` is a compile-time constant threaded through every cluster-dynamics template, `Library/Materials/Zr4.txt` can no longer be run from this branch at all. That is why the `Zr3d_ghoniem` material definition is a *new* file rather than an edit of `Zr4.txt`, which is left untouched.

Cost. The immobile DOF vector is 8 per node against 4 mobile, so the stored state per node roughly triples. The immobile solve itself is nodal and matrix-free, so this is memory, not time.

Change 2. The immobile time integration scheme

With the families created, an integrator for them had to be written. The scheme adopted is nodal collocation with sub-cycled, fused semi-implicit updates. The sibling `MoDELib-fullCD` branch made different choices at each of the three decision points, and is set beside it here because the contrast is what makes the reasoning legible — not because one was derived from the other:

	MoDELib-fullCD (sibling)	this branch
Rate	Galerkin: assembled at quadrature points, then L^2 -projected through the consistent mass matrix (CG to 10^{-4} , <code>SpatialODESolver</code>)	nodal collocation; no mass matrix
Time update	explicit Euler <code>dof += rate*dt</code> , then <i>sequential</i> implicit loss factors $(1 + \Delta t \lambda_k)^{-1}$ per channel	one <i>fused</i> semi-implicit update $\frac{n + \Delta t \dot{n}_{\text{nuc}}}{1 + \Delta t \lambda}$
Sub-cycling	none — one step of <code>dtMax</code>	<code>nSub</code> = 20 per dose step

Why. Three reasons, in order of weight. (i) The loop equations at a point depend only on that point’s own state and the local mobile field — there is no spatial operator in them — so the mass-matrix projection buys consistency of a quantity that has no spatial coupling to be consistent

about. (ii) Sequential per-channel loss factors do not commute, so the answer depends on the order the channels are applied; a fused update does not. (iii) One step of `dtMax` is not enough: at $G = 10^{-7}$ dpa/s a dose step is $\sim 7 \times 10^{19}$ MoDELib time units, and several channels have time constants far below that.

Code. `solveImmobileClusters()` in `src/ClusterDynamics/ClusterDynamicsFEM.cpp` — an *empty function body* in MoDELib2-NNL — is written as a node loop with an inner sub-step loop. DOF layout is `index = node*iSize + component`, with $[0, n_F)$ densities and $[n_F, 2n_F)$ contents. No mass matrix, no linear solve, no assembly: the immobile step is matrix-free.

Cost. This is the choice that makes the whole later coupling affordable. Collocating at nodes avoids a conjugate-gradient solve per dose step altogether; the 20 sub-steps cost 20 evaluations of an algebraic update at each node, which is arithmetic with no memory traffic beyond the node's own 8 DOFs. Measured later against the CVODE march, this nodal integrator is first-order semi-implicit and *has* to be: the measured stiffness puts its step 46 000 times beyond the forward-Euler stability limit. That measurement also corrected a description of it, common in earlier notes, as explicit Euler — it is not, and could not be.

3 Nucleation

Change 3. Cascade-borne loop nucleation, Eqs. (36)/(51)

Loops are seeded from the displacement cascade at $\dot{n}_{\text{nuc}} = G_0 \varepsilon_k / n_{\text{nuc}}(k) / \Omega$, with the matching content source $G_k = G_0 \varepsilon_k$, where ε_k is the family's share of the cascade (`loopCascadeFractions`) and n_{nuc} is the number of defects a newborn loop carries (`nNuc`).

Why. This is the 0-D's `nucleation_rate_iL/aiL`, and it must be reproduced term for term if the two codes are to agree.

Code. New parameters `loopCascadeFractions` and `nNuc` in `ClusterDynamicsParameters`, each guarded by `iSize>0` and converted to MoDELib units on read.

Cost. Negligible — two multiplications per family per sub-step.

Change 4. Homogeneous SIA clustering as a second nucleation source

$\langle a \rangle$ loops are also nucleated by interstitial clustering reactions whose product falls off the end of the mobile ladder. The channels are selected automatically by a new `getLoopNucChannels()`, which takes the pairs (a, b) from the existing `reactionMap` that (i) have reactants of the same polarity, and (ii) have a product size $m_a + m_b$ carried by *no* mobile species. For $\{v, i, 2i, 3i\}$ that selects $i+3i$, $2i+2i$ and $2i+3i$, and rejects $i+i \rightarrow 2i$, $i+2i \rightarrow 3i$ and every vacancy-bearing pair.

Why. $\langle a \rangle$ loop density measured $0.53\times$ the 0-D at 30 dpa and flat in dose, while the $\langle c \rangle$ density matched to 0.1% at every dose — so the defect was specific to the interstitial families. The 0-D nucleates $\langle a \rangle$ loops from *two* sources, and only the cascade one was wired:

source	loops/atom/s	share
cascade $G\varepsilon_{iL}/n_{iL,\text{nuc}}$	7.0429×10^{-13}	61.6%
clustering $R_{i,3i} + R_{2i,2i}$	4.3969×10^{-13}	38.4%

Cascade-only nucleation therefore predicts a density ratio of 0.62 against the 0.53 measured — the right size and sign for the entire discrepancy. The interstitials those reactions consume were simply disappearing: with no $4i$ species to receive the product, `getR2()` debits the reactants and credits nothing. That is the same truncation that raises `Warning: Sum of R2 is not zero` at start-up, which had until then been recorded as benign. It is not benign; it is a nucleation source.

Code. New member `loopNucChannels` and method `getLoopNucChannels()`. Taking the flux from the reaction network the mobile solve *already uses* makes the split mass-conserving by construction: the loops gain exactly what the mobile field loses. Two traps had to be navigated. (i) `getR2()` writes $-K_{ab}$ into *both* `R2[a](a,b)` and `R2[a](b,a)`, so $c^T R_2 c = -2K_{ab}c_a c_b$ — but the residual is

assembled as $R2*(0.5*mobileClusters)$, the quadratic-form factor that makes $bWF_R2 = R2*c$ its exact Jacobian, so the per-species loss rate is $K_{ab}c_a c_b$ and not twice that. (ii) Cascade-borne loops are born at $nNuc$ defects while clustering-borne embryos carry only the $|m_a| + |m_b|$ atoms of the event that made them, so the two sources must *not* be divided by the same number. The family split is in proportion to `loopCascadeFractions` within each polarity, which reproduces the 0-D's own f_{na}/f_a split exactly and generalizes to any number of families.

Cost. One pass over a three-entry channel list per node per dose step, hoisted out of the sub-step loop because the mobile field is frozen during it. Immeasurable.

Status. Verified against the 0-D jump frequencies $\omega_m = z_c \nu \exp(-E_m/kT)$ at 573 K:

channel	Zr3d_ghoniem K [1/s]	ZrMicro	error
$i + 3i \rightarrow 4i$	5.0535398×10^7	$\omega_i + \omega_{3i} = 5.0535398 \times 10^7$	4×10^{-6}
$2i + 2i \rightarrow 4i$	9.8765170×10^2	$4\omega_{2i} = 9.8812394 \times 10^2$	0.05%
$2i + 3i \rightarrow 5i$	4.9382578×10^2	$\omega_{2i} + \omega_{3i} = 4.9406197 \times 10^2$	0.05%

Effect at 30 dpa, median interior: $\langle a \rangle$ loop density $0.533 \rightarrow \mathbf{0.704}$, content $0.620 \rightarrow \mathbf{0.784}$, defects per loop $1.164 \rightarrow \mathbf{1.114}$, all as ratios to the 0-D. The $\langle c \rangle$ density was unaffected at 0.999, as it must be.

4 Growth by absorption

Change 5. Absorption growth with the diffusional-anisotropy bias, Eqs. (41)–(45)

Each family grows by absorbing mobile defects of like polarity and shrinks by absorbing the opposite kind: $\dot{g}_k = \phi_k^{\text{like}} - \text{gate} \cdot \phi_k^{\text{opp}}$, with $\phi_k = \sum_m \bar{D}_m Z(\text{row}(k), m) S_k c_m |m_m|$ — the size weight $|m_m|$ being what makes a $2i$ cluster deliver two interstitials.

Why. This is the growth flux of the 0-D's `loop_growth_rate_*` methods.

Code. New method `loopDADbias()` returning the Eq. (15) capture efficiencies $Z_c = Z^0 p$, $Z_a = Z^0(p + p^{-2})/2$, row 0 for $\langle c \rangle$ and row 1 for $\langle a \rangle$, from two new fitted parameters `dadAnisotropy` (p_m) and `dadZ0` (Z_m^0). The orientationally averaged diffusivity $\bar{D}_m = (\det D_m)^{1/3}$ is formed once per solve.

Cost. $n_F \times m_{\text{size}}$ multiply-adds per node per sub-step; the dominant arithmetic in the immobile step, and still trivially cheap.

Status. The DAD reconciliation is a *fitting* form, not a generated one — see Part VIII.

Change 6. The minimum-stable-size shrink gate, Eq. (91)

The shrinking channel is switched off smoothly for loops below the minimum stable radius: a cubic Hermite gate in $x = (r_k/r_{\min} - 1)/0.3$, zero at $x \leq 0$ and one at $x \geq 1$.

Why. Below $r_{\min}$ a loop is not a stable sink; letting it shrink further is unphysical and drives the radius to zero.

Code. Reads the existing `r_min` key — but *that key was never converted from metres*. It was read raw in SI and compared against radii from `rloop()/rpyr()`, which are in units of b . The gate was therefore permanently wide open and the change had no effect until `r_min` was divided by `b_SI` on read. The conversion is safe because `r_min` is used only by the new code.

Cost. One branch and a cubic per family per sub-step.

Change 7. Loop densities stored per b^3 , not per atom

The immobile number-density DOF is stored as loops per b^3 ; the content DOF is a defect volume fraction.

Why. `clusterRadius()`, `clusterDensity()` and `sigmoid()` all form $n = c/(N\Omega)$ — i.e. they *expect* N in loops per b^3 . The new integration was accumulating loops per atom.

Code. `nucRate` is divided by Ω , `Nvol` is $n(k)$ directly, and the like-loop spacing is $d_{LL} = (1/N)^{1/3}$. Content needs no conversion: a volume fraction is numerically equal to the 0-D per-atom content.

Cost. None. But the consequence of getting it wrong was not cosmetic: the helpers read the mean loop size as $2.82\times$ too large, which was enough to push the $\langle c \rangle$ family across the bi-pyramid $\rightarrow$ loop sigmoid onto the wrong branch. A units error in a quantity that feeds a branch condition changes which physics runs, not just by how much.

5 Annealing and thermal emission

Change 8. Vacancy-loop dissolution integrated implicitly

The n/τ_{vL} and c/τ_{vL} dissolution channels (Eqs. 38/53) were moved from the explicit source term into the implicit denominator.

Why. $\langle c \rangle$ loops were collapsing onto the positivity floor: n_{vL} pinned at 10^{-30} while $\langle a \rangle$ loops nucleated and grew normally. The cause was integration, not physics — $\tau_{vL} \approx 9.95 \times 10^{16}$ MoDELlib time units against a sub-step of 3.48×10^{18} , so **the decay time is 35 times shorter than the step**. Explicit Euler gives $n(1 - 35) < 0$, clamped every iteration.

The confirmation that this was integration and not a modeling error is the analytic balance: $n^* = \dot{n}_{\text{nuc}}\tau_{vL} = 1.79 \times 10^{-8}$, against the 0-D's $C_{vL} + C_{avL} = 1.78 \times 10^{-8}$. The physics was already right.

Code. The linear losses are collected into a single coefficient `lossN` and applied as $(n + \Delta t \dot{n}_{\text{nuc}})/(1 + \Delta t \text{lossN})$.

Cost. One division instead of one multiplication. The scheme becomes unconditionally stable in this channel and returns $n^* = \dot{n}_{\text{nuc}}\tau_{vL}$ exactly in the limit $\Delta t/\tau \rightarrow \infty$ — so the sub-step count is set by accuracy in the *other* channels rather than by stability in this one. Verified: $n_{vL} = 1.79161 \times 10^{-8}$.

Change 9. Annealing releases each loop's *birth* content, not its mean

The content sink from vacancy-loop dissolution was changed from c/τ_{vL} (proportional to content) to $n_{\text{nuc}} n \Omega/\tau_{vL}$ (proportional to *density*).

Why. $\langle c \rangle$ loop content came out 350 times below the 0-D, with a mean size $m_c = 487$ against $n_{\text{nuc}} = 400$ — i.e. pinned at the nucleation size — versus 1.7×10^5 in the 0-D. ZrMicro's `annealing_content_vL()` returns $n_{vL,\text{nuc}} C_{vL}/\tau_{vL}$: each dissolving loop releases the content it was *born* with, because τ_{vL} describes the dissolution of still-small embryo loops, while loops that survive and grow are stable. Removing the *mean* content instead caps the mean size at $\sim n_{\text{nuc}}$, which is exactly what was observed.

Code. A separate `contentSink` term, explicit (it is proportional to n , not to c , so it cannot enter the c -implicit denominator), added to the content source with a negative sign.

Cost. Arithmetically identical. The self-check is the valuable part: at number saturation $n^* = \dot{n}_{\text{nuc}}\tau_{vL}$, so the release equals $n_{\text{nuc}}\dot{n}_{\text{nuc}}\Omega = G_k$ and *exactly cancels the cascade content seed*, leaving net content growth flux-driven — precisely what the 0-D docstring describes. A term that is right in form can be checked against an identity like that; one that is right only in magnitude cannot.

Status. $\langle c \rangle$ content moved from $350\times$ low to within 20–26% of the 0-D.

Change 10. Thermal emission written as a coefficient, not a rate

The Eq. (55)–(56) single-vacancy emission term was recast from an absolute rate into an equivalent first-order loss coefficient.

Why. As an absolute rate it could not enter the implicit update at all, which left the stiffest available channel explicit.

Code. Per loop, $\alpha_v = 2\pi r D_v/\Omega \cdot e^{-E_b/kT}$ (the convention of `getR1()`); the content lost per unit volume is $\alpha_v N \Omega = S_k D_v e^{-E_b/kT}$, the Ω 's cancelling. Since that does not scale with c , it is divided by c to form an equivalent first-order coefficient and added to `lossC`.

Cost. One extra division per vacancy family per sub-step. It buys the ability to integrate the term implicitly, which is the point.

6 Coalescence

Change 11. Geometric coalescence in two channels, Eqs. (97)–(102)

Two Avrami-type coalescence channels were implemented:

$$\begin{aligned} \text{loop-loop: } \phi_{LL} &= 1 - \exp\left(-\kappa_{LL} \frac{4}{3} \pi r_{LL}^3 N\right), & \nu_{LL} &= c_{LL} v_{\text{abs}} N^{1/3} \\ \text{loop-network: } \phi_{LN} &= 1 - \exp\left(-\kappa_{LN} \pi r_{LN}^2 \rho_N\right), & \nu_{LN} &= c_{LN} v_{\text{abs}} \sqrt{\rho_N} \end{aligned}$$

with $r_{LL} = \min(r_k, N^{-1/3})$, $r_{LN} = \min(r_k, \rho_N^{-1/2})$, and the climb speed taken from the *absorbed* flux, $v_{\text{abs}} = \phi_k^{\text{like}} / (2\pi r_k N)$.

Why. These are the 0-D’s coalescence channels, and they are the dominant loop *removal* mechanism — without them the loop density grows without bound in dose.

Code. New parameters `cLL`, `cLN`, `kappaLL`, `kappaLN`, `rhoNetwork`. **The two channels are not symmetric**, and the asymmetry is physical: the like-loop channel merges two loops into one, so it removes number but *conserves content*; the loop–network channel absorbs a loop into the dislocation network, so it removes *both*. Hence `coalN` = `nuLL*phiLL` + `nuLN*phiLN` but `coalC` = `nuLN*phiLN` alone.

Taking the climb speed from the absorbed flux rather than the net one matters at saturation, where the net flux passes through zero while coalescence is still active; v_{abs} stays positive there.

Cost. Two exponentials, a cube root and a square root per family per sub-step — the most expensive arithmetic in the immobile update, and still far below the cost of a single sparse matrix–vector product.

Change 12. Coalescence moved into the implicit update

Both coalescence channels were reclassified as first-order loss *coefficients* and moved into the implicit denominator.

Why. The same stiffness failure as Change 8, in the channel that had been left explicit. `nuLN*phiLN*dt_sub` reaches ~ 3.6 , so each sub-step overshoots and clamps to the floor. The signature was unmistakable: $\langle a \rangle$ mean loop size $m_a = 5042$ at 1 dpa, then *exactly* 300.0 — = n_{nuc} — at 2 dpa. Content collapsing $11\times$ while density kept climbing is not a physical trajectory; it is a population being reset to its nucleation state every step.

Code. Eq. (97) removes loops in proportion to n and content in proportion to c , so both are already first-order coefficients and the move is exact, not an approximation. They are added to `lossN` and `lossC` respectively.

Cost. No arithmetic change. $\langle a \rangle$ loop density went from $3\text{--}4\times$ the 0-D to within $\sim 30\%$.

Change 13. $\langle a \rangle$ Burgers vectors given as lattice components

`immobileSpeciesBurgers` was corrected from Cartesian to lattice components: $(1, 0, 0)$, $(0, 1, 0)$, $(-1, 1, 0)$ for the three $\langle a \rangle$ families and $(0, 0, 1)$ for $\langle c \rangle$.

Why. At *zero* applied stress the three prismatic families must be identical by symmetry. They measured $n_{a1}, n_{a2}, n_{a3} = 2.84 \times 10^{-7}, 2.32 \times 10^{-7}, 3.16 \times 10^{-7}$.

Code. The columns of `immobileSpeciesBurgers` are multiplied by `latticeBasis` on read — they are lattice components, and Cartesian ones had been supplied. The resulting magnitudes were $|b| = 1.0, 0.753, 1.197$ instead of all 1.0. After the fix, `immobileSpeciesBurgersMagnitude`: 1.633, 1.0, 1.0, 1.0.

Cost. None.

Status. This established a **standing test**: at zero applied stress the three $\langle a \rangle$ family densities must agree. It has been checked on every run since.

Part II

The mobile solve, and the coupling between the two

The immobile families do not evolve in isolation: they absorb from the mobile field, and the mobile field must see them as sinks. Half the changes in this part exist because turning on the immobile side exposed defects in the mobile side that had been invisible while the immobile populations were zero.

Change 14. `ImmobileSinks.h` — the sink the mobile species see

A new `EvalFunction`, mirroring the existing `SecondOrderReaction`, supplying the loop absorption sink that the source had marked `// Missing immobile sinks`. For each mobile species m it assembles

$$k_m^2 = \sum_k Z(\text{type}(k), m) S_k, \quad S_k \text{ from } \text{clusterDensity}() = 2\pi r_k N_k$$

and returns $-k_m^2 \bar{D}_m$ on the diagonal, matching the sign convention of `getR1()`.

Why. Without it the coupling is one-way: loops grow by absorbing mobile defects that are never removed from the mobile pool. That is not a small error at dose — it is a violated mass balance that grows with the loop population.

Code. New file `include/ClusterDynamics/ImmobileSinks.h`, wired into the mobile Newton system in `ClusterDynamicsFEM.cpp`: `bWF_RI` into the Jacobian and `lWF_RI` into the residual, alongside the existing `R1/R2` terms.

Two sinks are deliberately *absent* from this file. The grain boundary is imposed spatially by the Dirichlet boundary condition on the mobile species and must not be double-counted here. The network dislocation sink reuses the existing `otherSinks_SI` plumbing, which required no new code at all.

Cost. One more bilinear weak form in the Newton assembly — of the same order as `R1`. This is a real per-iteration cost on the fast solve, and it is one of the terms later hoisted out of the Newton loop (Change 43).

Change 15. `loopSinkScale` — reproducing the 0-D sink convention exactly

An optional material key, default `(1,1,1,1)`, multiplying the loop sink strength: `loopSinkScale = (0.291528, 0.792317, 0.792317, 0.792317)`.

Why. `ZrMicro`'s loop absorption is *not* the purely geometric $S_k = 2\pi r_k N_k$, and it departs from it in two independent ways.

(a) *One radius prefactor for all four families.* `input_data.py` derives both $l_a = \sqrt{\Omega/\pi b_a}$ and $l_c = \sqrt{\Omega/\pi b_c}$, but `reaction_rates.loop_absorption()` uses l_c/l for *every* family — l_a is computed and never used anywhere in the codebase. The 3-D builds r_k from each family's own $|b_k|$, so each needs $\sqrt{|b_k|_{\text{MoDELib}}/b_c^{\text{ZrMicro}}}$: for $\langle c \rangle$, $\sqrt{5.279467/5.15} = 1.012495$; for $\langle a \rangle$, $\sqrt{3.233/5.15} = 0.792317$.

(b) *A fitted Q on the $\langle c \rangle$ channel only:* $1.012495 \times 0.287931 = 0.291528$.

Code. Applied *identically* in `ImmobileSinks.h` (the sink seen by the mobile species) and in `solveImmobileClusters()` (the growth flux it causes) — mirroring the 0-D, where `loop_absorption()` and the `loop_growth_rate_*` methods share one prefactor. Applying it in only one place would break the mass-conserving coupling between them, which is worth stating because it is the kind of asymmetry that produces a slow conservation drift rather than an obvious failure.

Cost. One multiplication.

Change 16. `Eb` declared before `R1` — an inherited upstream bug

The declaration of `Eb` was moved ahead of `R1` in the class.

Why. C_{2i} and C_{3i} came out at $\sim 10^{-17}$ against the 0-D's $\sim 2.5 \times 10^{-11}$ — six orders low. `getR1()` reads `Eb(k)` to build the cluster dissociation rates, but `Eb` was declared *after* `R1`. C++ initializes

members in **declaration** order, not in the order of the constructor’s initializer list, so this was an uninitialized read. It evaluated to $\exp(0) = 1$ instead of $\exp(-E_b/kT)$, inflating the $2i/3i$ dissociation rate to the bare attempt frequency.

The evidence was arithmetic rather than circumstantial: $R1(2,2) = -5.0535 \times 10^7$ 1/s where the network sink alone gives -20.86 ; $R1(3,3)/R1(2,2) = 0.4802 = p(1,2)/p(1,1)$ exactly, so *both* dissociation rates had lost their Boltzmann factor; and the predicted C_{2i} under the bug, $G_{2i}/\alpha = 9.89 \times 10^{-18}$, matched the observed 1.06×10^{-17} .

Code. A declaration move, with the reasoning recorded at the declaration site so it cannot be undone by a later tidy-up.

Cost. None. $R1(2,2) \rightarrow -21.05$ 1/s; $C_{2i} + 1.2\%$ and $C_{3i} - 2.3\%$ against the 0-D.

Status. This is a bug in the original code, not in the Zr3d_ghoniem changes. It was invisible in Po’s calibration because $E_b = 5.0$ eV for every species makes $\exp(-5/kT) \sim 10^{-44}$, negligible whether or not the factor is applied. Giving $2i$ its physical 0.957 eV is what exposed it. It affects any MoDELib run with a non-negligible cluster binding energy and is worth reporting upstream.

Change 17. Clusters transport at the monomer diffusivity

E_m for $2i$ and $3i$ was set to the monomer value, and the ω_{2i}/ω_i factor moved into the reaction prefactors instead.

Why. Setting $E_m^{2i} = 1.36292$ eV drops D_{2i} to $\sim 5 \times 10^{-6}$ of D_i , and `FixedDirichletSolver` failed. was thrown from `compute()` — the factorization, not the solve. The mobile solve is a *pure steady-state* (QSSA) system: `dmBWF` is diffusion only, with no mass term, so the $2i/3i$ block had nothing on its diagonal but a vanishing Laplacian. The matrix was singular.

The fix is also *more* faithful to the 0-D, which is the part worth recording. ZrMicro is inconsistent about which frequency it applies where, but it is unambiguous: `R_2i_s/R_3i_s` (network sinks) and `flux_i` (loop absorption) all use ω_i , and ω_{2i} appears *only* in the mobile–mobile cluster reactions.

Code. Material file change plus the reaction-prefactor derivation of Change 18. The network-sink and loop-absorption terms now reproduce the 0-D exactly, which they did not before.

Cost. Removes a singular block, which is the difference between a solve and no solve.

Change 18. The reaction prefactors derived and verified

All ten entries of `reactionPrefactorMap` recomputed from

$$p_{ab} = f_{ab} \frac{z_c}{a^2} \frac{\Omega}{4\pi(r_a + r_b)} \frac{D_a^{0D} + D_b^{0D}}{D_a^{3D} + D_b^{3D}}$$

Why. MoDELib forms $K_{ab} = p_{ab}4\pi(r_a + r_b)(D_a + D_b)/\Omega$ in `getR2()`, while ZrMicro forms $K_{ab} = f_{ab}(\omega_a + \omega_b)$. Equating them is the only way the two reaction networks can agree. The last ratio is unity except for pairs involving $2i/3i$, where it restores the ω_{2i} -based rate now that those species transport at the monomer diffusivity (Change 17).

Code. Material file only.

Cost. None.

Status. Confirmed empirically. MoDELib prints the assembled second-order interaction matrices in 1/s at start-up:

entry	MoDELib	ZrMicro	agreement
(v, i)	2.526876×10^7	$0.5(\omega_i + \omega_v) = 2.52560 \times 10^7$	0.05%
(i, i)	-2.021161×10^8	$-4\omega_i = -2.020216 \times 10^8$	0.05%
$(v, 2i)$	6.940722×10^3	$\omega_{2i} + \omega_v = 6.93473 \times 10^3$	0.09%

This also settles a factor-of-two ambiguity that could not be resolved by reading the code: `SecondOrderReaction.h` documents the rate as $\dot{c} = \frac{1}{2}R_2cc$ while `getR2()` fills both (a, b) and (b, a) , and it was not clear whether the $\frac{1}{2}$ survives into the weak form. The match above shows it does.

Change 19. `atomicVolume_SI` — an atomic volume independent of the cell volume

A new optional material key, read by a new static `getClusterAtomicVolume()`.

Why. `Polycrystal::Omega` is $\det(\text{latticeBasis}) = 1.41421 b^3$ for HEX — the volume of a cell containing **two** atoms, not the atomic volume the cluster-dynamics equations assume. Every content $\leftrightarrow$ radius $\leftrightarrow$ density conversion inherited the error.

Code. Made *optional* deliberately: material files that omit the key keep the previous behavior, so `Polycrystal` — which the elastic and dislocation-dynamics sides also use — is left alone. This is the one place where a global correction would have reached beyond cluster dynamics, and it was scoped rather than applied globally.

Cost. None directly, but see Change 20, which this caused, and Change 31, which corrected the *value* much later.

Change 20. `mBWF/dmBWF` scale with `cdp.omega`, not `poly.Omega`

Why. Introduced by Change 19. `FluxMatrix` returns $-D/\text{cdp.omega}$, and the weak forms multiplied it back by `ddBase.poly.Omega`. Those two were the same number until the CD atomic volume became independently settable; afterwards the diffusion term carried a spurious factor of $\text{poly.Omega}/\text{cdp.omega} = 1.41421/0.355105 = \mathbf{3.98}$ against every reaction term.

Code. Use `cdp.omega` in both weak forms, restoring the exact cancellation so the flux is $-D\nabla c$.

Cost. None. Found by reading the constructor while chasing the non-convergence of Change 21 — a factor of four on the diffusion operator is a plausible contributor to that, and this is a reminder that a correction in one place (an atomic volume) can silently unbalance a cancellation somewhere else entirely.

Change 21. The mobile fixed-point iteration: five solver controls, and a diagnosis

Five optional `DD.txt` scalars were added to the mobile fixed-point loop, all defaulting to the historical behavior except the iteration cap: `mobileSolverTolerance` (10^{-5}), `mobileSolverMaxIterations` (200; 0 = unlimited, the old behavior), `mobileSolverRelaxation` (1.0), `mobileSolverErrorMode` (0), `mobileSolverClampInLoop` (1).

Why. `solveMobileClusters` iterates `while(cError > cTol)` with **no iteration cap and no stagnation test**. That code is inherited verbatim from `MoDELib2-NNL`, and the sibling `MoDELib-fullCD` carries the identical loop, so it is not a defect introduced here. It is safe only while the iteration always converges. Solving the mobile field against an immobile state supplied by an external march, `cError` falls from 5.8×10^5 to 1.3×10^{-4} in nine iterations and then *stagnates*, oscillating between 4.7×10^{-5} and 1.2×10^{-4} . With no cap, `DDomp` never returns.

The cause is a clamp added *here*, not anything inherited. Neither `MoDELib2-NNL` nor `fullCD` has a mobile clamp at all; this branch inserted `clampMobileClusters()` between the Newton update and the error test, which turns the iteration into a *projected* Newton method.

The precise failure is worth stating, because the obvious explanation is wrong. The active set does *not* oscillate: an instrumented run shows it freeze at 7090 DOFs by iteration 3 and never change again. What fails is that the Newton matrix is the *unconstrained* Jacobian — it does not know any DOF is pinned. Each iteration it computes an increment for all DOFs including the pinned ones; the projection then discards exactly those moves, but the increment for the *free* DOFs was computed assuming the pinned ones would move. The free DOFs respond every iteration to a coupling that is immediately cancelled, and that inconsistency does not decay. Solve-then-project is simply not a consistent method for a constrained system.

Code. Each iteration now prints the iteration number, which species attains the maximum, and how many DOFs the floor moved. On hitting the cap the *best* iterate seen is restored, not the last.

Cost. Measured from an identical saved immobile state (24 115 CD nodes, 96 460 mobile DOFs):

configuration	iters	converged	final cError	wall
clamp in loop, undamped (as shipped)	30 (capped)	no	6.4×10^{-5} , cycling	392 s
clamp deferred out of the loop (fullCD)	5	yes	6.83×10^{-7}	112 s
clamp in loop, damped $w = 0.5$	30 (capped)	not yet	9.4×10^{-5}	400 s
clamp in loop, damped $w = 0.7$	20	yes	8.2×10^{-6}	283 s
clamp in loop, errorMode=1	40 (capped)	no	5.1×10^{-5}	492 s
clamp in loop, cTol=1e-4	19	“yes”	9.2×10^{-5}	258 s

Deferring the clamp restores **quadratic** convergence — $2.95 \times 10^4 \rightarrow 1.23 \times 10^3 \rightarrow 6.77 \times 10^1 \rightarrow 5.73 \times 10^{-2} \rightarrow 6.83 \times 10^{-7}$ — i.e. a genuine Newton method, and it is faster than the solve has ever been: 3.5× against the shipped configuration.

The accuracy consequence is larger than the speed one. Maximum relative difference in C_i against the converged field, over the 22 344 nodes above the floor: damped $w = 0.7$, **0.19%**; **errorMode=1**, 21.1%; **cTol=1e-4**, 49.9%; baseline, **53.6%**. Two configurations that *report* success are wrong by half. Relaxing the tolerance is not a fix; it declares victory on the bad field.

Status. Both routes must use `mobileSolverClampInLoop=0` — the coupled march sets it in `modelib_qssa`, and the standalone comparison driver in `setup_standalone.FAST_STEP_SETTINGS`. Left at the default on one side only, a route-to-route comparison measures this solver defect rather than the physics.

Change 22. `useImmobileSolver` — running the fast step alone

A new `DD.txt` scalar, default 1 when the key is absent so every pre-existing case is unaffected, making the `solveImmobileClusters()` call conditional.

Why. `ClusterDynamicsFEM::solve()` already *is* the two-time-scale split: `solveMobileClusters()` is the fast step (steady $C_M^*(x)$, no time derivative anywhere) and `solveImmobileClusters()` is the slow one. To let an external driver own the slow step — which is what the whole coupled march is — the second call must be suppressible.

Code. With `useImmobileSolver=0`, `DDomp` performs the fast step and passes the immobile field through untouched. Because `runSingleStep` writes output *before* incrementing `runID`, a run with `startAtTimeStep=0` and `Nsteps=1` reads `evl_0.txt` and overwrites the same file — an in-place round trip, which is exactly the interface a Python driver wants.

`modelib_qssa.py` asserts on the immobile solver SKIPPED line, so a binary predating this flag fails *loudly* rather than silently advancing the immobile field twice. That assertion is the only thing standing between a stale binary and a run that looks fine and is wrong by a factor of two in dose.

Cost. This is the enabling change for the entire coupled architecture of Part IV.

Part III

The material file and the verification case

Change 23. `Library/Materials/Zr3d_ghoniem.txt` — a new material definition

Everything outside the `# ClusterDynamics` section (elasticity, mobility laws, gamma surface) is inherited unchanged; original values are preserved in comments in the file itself. The changed values:

Key	Zr4 (Po)	Zr3d_ghoniem (Zr-Micro)	Basis
atomicVolume_SI	<i>absent</i>	1.2e-29	ZrMicro <code>physical_props</code> [<code>'Omega'</code>]
mobileSpecies	0.01	1.0	the 0-D loses defects only to the in-cascade cluster fractions
SurvivingEfficiency			
mobileSpecies	1.0 0.6	0.995 0.97788714	$G_v/G = 1 - \varepsilon_{vL}$, $G_i/G = 1 - \varepsilon_{2i} - \varepsilon_{3i} - \varepsilon_{iL}$, etc.
CascadeFractions	0.3212 0.0788	0.005 0.00333333	
mobileSpecies	1.5	1.6	0-D E_F^v ; gives $C_v^{\text{eq}} = 8.45 \times 10^{-15}$ at 573 K
EnergyFormation_eV			
mobileSpecies	0.9 / 0.09	1.2 / 0.759101	0-D E_m^v , E_m^i ; clusters at the monomer value (Change 17)
EnergyMigration_eV	/ ...	($\times 3$), isotropic	
mobileSpeciesD0_SI	4.9639e-10, 8.1882e-10	5.21645e-7 all, isotropic	$D_0 = a^2 \nu$, $a = 3.23 \times 10^{-10}$ m, $\nu = 5 \times 10^{12}$ s $^{-1}$
reactionPrefactorMap	2.856, 1.0, 0.0	all ten recomputed	Change 18
Eb_eV	5.0 5.0 5.0 5.0	5.0 5.0 0.957033 2.0	index 0 $\rightarrow$ loop emission; 2, 3 $\rightarrow$ 0-D E_b^{2i} , E_b^{3i}
otherSinks_SI	0 0 0 0	1.88e14, 1.8988e14 $\times 3$	network dislocation sink, $k_v^2 = \rho_N$, $k_i^2 = Z_N \rho_N$

Why. Every one of these is a term the 0-D carries; leaving any of them at the Po calibration means the two codes are solving different models and no comparison between them means anything.

Code. A new §2.2 block adds `immobileSpeciesVector`, `immobileSpeciesRelRelaxVol`, `immobileSpeciesBurgers`, `alpha_bp`, `delVPyramid`, `w0`, `n_s`, `evc`, `Nvmax`, `nmin`, `nmax`, `r_min`, `distanceFactor`, `loopCascadeFractions`, `nNuc`, `cLL`, `cLN`, `kappaLL`, `kappaLN`, `tau0_vLoop_SI`, `Ea_vLoop_eV`, `rhoNetwork_SI`, `dadAnisotropy`, `dadZ0`. The coalescence coefficients, nucleation sizes and dissolution constants are the fitted 0-D values from run 20260622_144021_7959445.

The migration-energy row order matters and is easy to get wrong: components are [11 12 13 22 23 33], and the off-diagonal D_0 entries are zero, so the tensors are diagonal.

Cost. None.

Status. `Zr4.txt` is **untouched** and carries no immobile keys — but that is a property of this one file and **not** evidence about any other branch. `fullCD's Zr4_Fitted.txt` and `Zr4_BMD19_nuc.txt` each carry a full immobile key set. `Zr4.txt's` missing keys have been cited as evidence about upstream in earlier notes; they are not.

Change 24. `tutorials/zrmicro_coupled/` — the verification case

A cluster-dynamics-only verification case: `useFEM=1`, `useDislocations=0`, `useClusterDynamics=1`, `dtMax = 6.95868964e19` (= 1 dpa at $G = 10^{-7}$), `outputFrequency=1`, `startAtTimeStep=0`, `absoluteTemperature=573`, non-periodic, zero target loop densities.

Why. A fixed, reproducible case is the only way to attribute a change in the answer to a change in the code.

Code. `clean_run.sh` refreshes the material file and regenerates `evl_0` on every invocation. Four separate traps made that necessary, and each is worth recording because each failed *silently*:

1. *microstructureGenerator never terminates.* It is the discrete-dislocation path: it inserts loops one at a time, and the log showed it had reached 2.34×10^{17} m $^{-3}$ against a target of 1.48×10^{21} — 0.016% of the way, needing $\sim 10^{11}$ discrete loops. Cluster-dynamics loop densities are not representable as discrete dislocations, which is precisely why they belong in the continuum fields. Fixed with `useDislocations=0` and `targetDensity=0`, giving an empty 20-byte `evl_0.txt` in about a second. DDomp still requires that file to exist even with dislocations off.
2. *Restart silently double-counted dose.* $\langle a \rangle$ loop content came out at exactly $3\times$ the expected value. `startAtTimeStep=-1` means “last available step”, and `evl_0.txt` serves double duty as

both the initial configuration *and* the runID 0 output, so a bare re-run resumed from the previous run’s accumulated dose. Diagnosed by $c_a^{\max} = 2.112 \times 10^{-3} = \text{exactly } 3 \times (G_0 \varepsilon_a \cdot 1 \text{ dpa})$.

3. *Stale material file.* `inputFiles/` holds a *copy* of the material file, made before the §2.2 block existed; the run died on does not contain line with `format immobileSpeciesVector=...`
4. *CRLF line endings produced a zero-node mesh.* `unitCube_15K.msh` checked out with CRLF and the `.msh` parser returned zero nodes *without complaint*, surfacing much later as a periodicity failure.

Cost. None, but the recurring hazard is worth stating once: **almost every failure in this stack exits 0** — the build script, DDomp with a missing `ev1_0`, a missing material key, a CRLF mesh, and the solver failures above. Verification has been by artifacts on disk and log tails, never by exit status. Later, `build.preflight()` was written to *run* each binary rather than trust its presence, for the same reason.

Part IV

The coupled architecture

With the 3-D immobile physics working, the question became how to drive it. The changes in this part are on the Python side and in the repository structure; they are what turned a verification case into a simulation code.

Change 25. A per-node field channel replacing the four-scalar handoff

The 0-D $\rightarrow$ 3-D handoff was replaced by a genuine field exchange through MoDELib3’s own cluster-dynamics configuration block.

Why. The previous route, `legacy/modelib_fem.py`, wrote *four scalars* per dose step into the microstructure-generator inputs, discarding all spatial structure. A spatially resolved code driven by a spatially averaged boundary condition is not a coupled calculation.

Code. `modelib_field.FieldBridge.write_immobile_field` writes the per-node immobile state into the `ev1` CD block; `modelib_qssa.MobileQSSASolver.solve` reads $C_M^*(x)$ back at the CD nodes. `legacy/modelib_fem.py` is kept, unused, only so older results stay reproducible.

Cost. The exchange is file-based and $O(\text{nodes})$; at 90 617 nodes it is a few seconds against a fast solve of several minutes.

Change 26. The fast solve moved *inside* the march loop

Why. This is the load-bearing correctness point of the coupling. If the mobile field is instead replayed from a previously recorded run, the loop is not an operator split at all: C_M never responds to the immobile state the march is building, so a seed away from quasi-steady state can never relax. A 0-D seed is always such a seed — it is spatially uniform and carries no boundary layer, while the true $C_M^*(x)$ is pinned to thermal equilibrium on every Dirichlet face.

Code. `coupling/march.py` alternates `qssa.MobileQSSASolver.solve` (DDomp with `useImmobileSolver=0`) with `run_immobile_step` (one OpenMP batch subprocess) every `fem_every` substeps.

Cost. This is where essentially all the wall time goes, and it is why Part VI exists. Splitting error is first order in the interval between fast solves — `fem_every` substeps — *not* in the dose-snapshot spacing, so `fem_every` is the accuracy dial. It mattered, therefore, that a value not dividing `substeps_per_interval` was silently skipping whole intervals; the configuration validator now rejects that.

Change 27. CVMODE at every quadrature point, with the mobile field frozen

The slow step integrates the immobile ODEs independently at every node with `freeze_mobile=1`, using CVMODE BDF with a dense 15×15 linear solver, dispatched through the 0-D solver's OpenMP `-batch_file` mode.

Why. This is the design point that makes the whole scheme tractable, and it is easy to misread the numbers as claiming something impossible. **A tightly coupled million-equation stiff system would be hopeless. This is not one.** It is 72 494 independent 15-dimensional systems, not one 1 087 404-dimensional one (500 nm case: 90 617 nodes, deduplicated $\times 1.25$).

The decoupling is legitimate rather than a cheat because **the only thing coupling neighboring points is diffusion of the mobile species**, and the slow step freezes those. Freeze the mobile field and the spatial coupling is gone by construction: a point's immobile ODEs depend only on its own loop populations and the mobile values pinned into its state vector.

Code. `rate_equations.ode_system(t, y)` takes a single point's state vector; no neighbor state enters it. Each point is one case of the batch mode, dynamically scheduled over 24 threads with a private CVMODE workspace each.

Cost. The cost consequence, for one implicit factorization:

	flops
split: $72\,494 \times 15^3$	2.4×10^8
coupled: $(1\,359\,255)^3$	2.5×10^{18}
ratio	1.0×10^{10}

Sparsity softens the coupled figure but does not rescue it — a 3-D FE Jacobian still factorizes at $\sim O(N^2)$ with fill growing as $O(N^{4/3})$.

Stiffness localizes the same way. Each point's system genuinely is stiff (~ 100 CVMODE internal steps per point per substep, which is why BDF), but stiffness cost scales with the *dimension of the coupled block*, not with how many independent blocks there are. A 15×15 dense LU is ~ 1100 flops.

The coupling did not vanish — it moved to where it is affordable. The tightly coupled problem still exists: it is the fast solve, 362 468 unknowns, **925 s** measured, against 28.3 s for an entire slow sweep over 90 617 points. What the split buys is *frequency*: `solveMobileClusters` has no time derivative, so the coupled problem is solved **8 times over the march** instead of at every timestep. The counterfactual, from the same measurements (925 s/5 Newton iterations = 185 s per linear solve):

~ 100 internal steps $\times$ 24 substeps $\div$ 5 steps per refactorization $\approx$ 480 factorizations
 $480 \times 185 \text{ s} \approx$ **25 h** — the 4-species mobile block *alone*, not all 19
measured slow side, all 24 substeps: **11.3 min**

Three things stacked: freezing the mobile field removes the spatial coupling; the coupled part has no time derivative so it is solved 8 times rather than thousands; and batch OpenMP plus deduplication (identical states solved once — $\times 1.25$ here, higher early on when much of the domain shares a state) spreads what is left over 24 cores.

The slow side is linear in node count, as this structure predicts: 9.3 s per substep at 27 720 nodes and 28.3 s at 90 617 — 3.35×10^{-4} versus 3.12×10^{-4} s/node.

Change 28. Configuration as a value: `SimulationConfig` and domain-keyed staging

The notebook was reduced to six control dicts, validated by `disloccluster_code.config.SimulationConfig.from_dicts`.

Why. A run whose parameters live in scattered module globals cannot be reproduced, and a validator that runs *after* an expensive stage costs a whole march.

Code. Unknown keys are rejected rather than ignored, and validation happens before anything expensive runs: a dose grid that does not increase strictly, a seed past the first snapshot, a boundary

layer thicker than half the domain, an `element_order` outside $\{1, 2\}$ (MoDELib's `SimplexReader` consumes only mesh element types 4 and 11).

Staging is keyed on the *domain*, not the run: `cfg.domain_key` hashes the geometry, mesh, boundary conditions, material and temperature — but *not* the dose grid — so changing only the doses reuses the staged case, its CD node set and its bootstrap.

One deliberate non-change: `coupling.march` still carries `DOSE_SEED`, `SUBSTEPS_PER_INTERVAL` and friends as module globals, read by `default_config()` *at call time*, so that callers which monkey-patch them still work. Turning those into dataclass field defaults would bind them at import and make a later assignment a silent no-op.

Cost. Re-running a dose sweep on a staged domain skips mesh generation and bootstrap entirely — minutes to hours, depending on the mesh.

Change 29. BOUNDARY made a real boundary-condition channel

`periodic_face_ids` goes into `polycrystal.txt`; the applied load goes into `ElasticDeformation.txt`.

Why. Previously the only available condition was Dirichlet everywhere.

Code. Stress is given in **MPa** and converted on staging: MoDELib normalizes stress by `mu_SI`, so the numbers in that file are multiples of the shear modulus, and writing MPa straight in would overstate the load by $\sim 33\,000\times$ for Zr.

Cost. None.

Change 30. Package consolidation and single-source path resolution

All importable code was gathered into `disloccluster_code/` at the repository root, installed editable, with every repository location resolved by `paths.py` from a `.disloccluster_root` marker.

Why. Four modules had re-derived their locations with `Path(__file__).parent.parent`, and that idiom broke *silently* when the package moved — `cpp_bridge` resolved its default `base_dir` that way and stopped finding `solver.exe` at all.

Code. `ZrMicro/py_utils/` remains as a compatibility shim: each old module aliases `sys.modules[__name__]` to its new home, so `from py_utils import paths`, `python -m py_utils.X` and the old notebooks all keep working, and `py_utils.X is disloccluster_code.... is true`.

Cost. None. `paths.describe()` now produces an OK/MISS report per location, and `paths.workbook_drift()` hashes the two input workbook directories and names any that have diverged — the same failure mode that once put the 0-D calibration five orders of magnitude off.

Part V

Physics corrections found by measurement

The changes in this part were all found the same way: a number was computed a second, independent way, and the two did not agree.

Change 31. The atomic volume corrected to the hcp Zr value

Ω was changed from $1.2 \times 10^{-29} \text{ m}^3$ to **$2.326553 \times 10^{-29} \text{ m}^3$** .

Why. The old value is $V_{\text{cell}}/4$ — four atoms in an hcp primitive cell that holds two. It implies $\sim 12.6 \text{ g/cm}^3$ for Zr, about half the true atomic volume. Two independent derivations from data already in the input files agree to **0.14%**:

route	value
lattice: $(\sqrt{3}/2)a^2c/2$, $a = b_a = 3.23 \text{ \AA}$, $c = b_c = 5.15 \text{ \AA}$	$2.326553 \times 10^{-29} \text{ m}^3$
mass density: $M_{\text{Zr}}/(\rho N_A)$, $\rho = 6520 \text{ kg/m}^3$	$2.3233 \times 10^{-29} \text{ m}^3$
former value ($V_{\text{cell}}/4$)	$1.2 \times 10^{-29} \text{ m}^3$

The lattice route is the one adopted, so that Ω stays consistent with the same b_a and b_c that the loop-radius prefactors $\sqrt{\Omega/\pi b}$ use.

Code. Changed in four places, which must stay equal: Simulations/input/Zr_input_parameters.xlsx (Physical_Properties!D5); ZrMicro/input/ — the same cell, copied byte for byte so that `paths.workbook_drift()` stays clean; MoDELlib3/Library/Materials/Zr3d_ghoniem.txt (atomicVolume_SI); and zerod/input_data.py, whose fallback default read 1.4×10^{-29} — a *third* number for one lattice constant. `InputData.check_atomic_volume()` runs on every `build_sim` and is what would catch a regression.

Cost. None computationally. But Ω is not a reporting factor: every number density is C/Ω , line density is $2\pi rC/\Omega$, and emission carries $\exp(\sigma\Omega/kT)$. Measured on the fitted set at 10 dpa: N_a 0.483 $\times$, N_c 0.516 $\times$, c_a 0.522 $\times$, C_v 1.047 $\times$, C_i 1.059 $\times$.

Status. The 28-parameter fit is now stale. The loop densities halve, and those parameters were fitted against experimental loop densities at the old Ω . The model now has the right lattice constant and a fit that no longer matches experiment. **A refit is required before comparing to data** and is planned separately. `calibration.LEGACY_OMEGA = 1.2e-29` reproduces a pre-correction run. **Runs made before this correction are not comparable to runs made after.**

Change 32. Conservation reinterpreted as an open-system balance

The six accumulators integrate a 0-D atom balance with **no transport term**: $d(\text{stored})/dt = \text{production} - \text{recombination} - \text{sink absorption}$. On a spatially resolved march that cannot close, and the residual is *not* solver error — it is the atoms that left through the Dirichlet surface.

Why. Two things force it: the domains here are Dirichlet over their whole surface unless faces are made periodic, and the slow step *freezes* the mobile species while production keeps accumulating into them (the mobile pool’s own balance is closed by the *fast* solve, which is where the flux to the boundary lives).

The magnitude settles it: the 200 nm march reports a residual of **−71% of cumulative production**. Nothing in a CVODE run at `rtol=1e-6` is 71% wrong.

Code. `post_process._calculate_conservation` also publishes `grain_boundary` (= −residual, positive when atoms have left), plus `mobile_change` and `immobile_change`. `post.volume_average` presents it as a physical channel because it *knows* the surface is absorbing — it reads `periodic_face_ids` from `config.json`. A standalone 0-D run leaves it as the numerical residual it genuinely is there (closing to 3×10^{-12}), and `plot_conservation` falls back to the old relative-error figure.

Cost. None. It is closure *by difference*, not an independent measurement — it inherits every other channel’s error, and its credibility rests on those being exact solver integrals rather than reconstructions. Which is why Change 35 exists.

Change 33. Conservation figures in defect counts, not fractions

The fate figures were changed from fractions of production to absolute defect counts, with $n_{\text{atoms}} = V/\Omega$, V from `fields.domain_volume` — the crystal, not its bounding box.

Why. A fraction hides the size of the system it is a fraction of, and the `*_fractions` figures had a “Sum (=1)” line that was reaching ~ 0.3 , because the boundary channel was missing from it.

Code. The 200 nm prism is 6.93×10^8 atoms, and at 10 dpa:

channel	interstitials	vacancies
produced	6.24×10^9	6.24×10^9
recombined	1.43×10^9 (22.9%)	1.43×10^9 (22.9%)
absorbed at network sinks	3.51×10^8 (5.6%)	3.20×10^8 (5.1%)
absorbed at grain boundary	4.45×10^9 (71.4%)	4.48×10^9 (71.9%)
stored in microstructure	4.71×10^6 (0.1%)	1.33×10^6 (0.0%)

Equal Frenkel production and equal recombination are exact; the storage asymmetry (4.7×10^6 against 1.3×10^6) is the bias-driven excess that drives growth. The “Sum (=1)” line is true again.
Cost. None.

Change 34. The accumulators are per-interval, not cumulative

`volume_average.cumulate_accumulators` was added to cumulative-sum the six accumulator columns along the dose axis before they are reported.

Why. The march restarts each interval’s integration from the marched state, so each snapshot’s accumulators hold that *interval’s* contribution, not the running total. Reporting them as if cumulative understates every channel by the history preceding the last interval.

Code. `build_results(..., per_interval_accumulators=True)`.

Cost. None.

Status. This was found *by* the independent boundary-flux check of Change 35 — exactly the job that check was added to do. An earlier hypothesis, that the discrepancy came from mixing the old and new Ω , was tested with LEGACY_OMEGA and **was wrong**: it barely moved the ratios.

Change 35. The grain-boundary channel measured a second, independent way

`post/boundary_flux.py` obtains the boundary flux without ever touching the accumulators. The fast solve returns a *steady* mobile field, so for each mobile species $0 = \nabla \cdot (D\nabla C) + R$, and by the divergence theorem

$$\int_{\Omega} R dV = \oint_{\partial\Omega} (-D\nabla C) \cdot n dA = \text{net outward flux.}$$

Why. Change 32 is closure by difference. A channel obtained by subtracting everything else from production is only as good as everything else, and a 71% number deserves a second opinion.

Code. R is exactly the 0-D right-hand side for the mobile components — that model has no diffusion, so its mobile equations *are* the reaction terms, and the grain boundary appears nowhere in it. That is why the volume integral of its mobile RHS measures the flux to the boundary. Atom weights are (1, 2, 3) for (C_i, C_{2i}, C_{3i}) and 1 for C_v , matching the stored channels.

Measured against closure on the 200 nm march:

dose	measured	closure	ratio
0.01	5 666 311	5 106 945	1.110
0.1	44 999 294	44 632 725	1.008
1	431 455 063	444 694 897	0.970
10	4 287 277 443	4 450 899 303	0.963

Agreement to 3.7% at 10 dpa — an independent confirmation, not an identity. The 11% at 0.01 dpa is the trapezoid over a coarse log grid where the rate falls 36% across the first interval, not a physics discrepancy.

Cost. Seconds. `march_report` calls it *guarded*, so a diagnostic can never cost a report that a multi-hour march earned.

Status. A surface integral inside MoDELib, where the FE gradients already exist, would be the gold standard. This needs no C++ change and works on runs that already exist, which is why it came first. On the 500 nm reference march the ratio sits at a stable 0.63–0.65; the leading hypothesis is Part VIII, item 4.

Part VI

Computational efficiency

Everything in this part leaves the answer unchanged. Where that was checkable bit for bit, it was checked bit for bit, and the check is reported alongside the speedup — a speedup obtained by changing the answer is not a speedup.

7 The 0-D solver

Change 36. An exact Jacobian by forward-mode automatic differentiation

The right-hand side was moved into a single scalar-type-templated core, so that the residual and an *exact* Jacobian are two instantiations of one definition. SUNDIALS’ difference-quotient Jacobian was retired.

Why. A difference-quotient Jacobian on a system spanning nine orders of magnitude in concentration is inaccurate where it matters most, and it costs n extra RHS evaluations per Newton matrix.

Code. One definition, two instantiations — so the Jacobian cannot drift out of step with the residual, which is the usual failure mode of a hand-written one.

Cost. At $n = 19$ the dense linear algebra was never the bottleneck, so this bought accuracy and robustness rather than wall time. Reported as measured.

Change 37. The six accumulators moved out of the implicit system

The conservation accumulators were recognized as pure quadratures and moved into CVODES quadrature variables.

Why. They are integrals of quantities the solver already computes; nothing depends on them, so they need not be inside the Newton block.

Code. The Newton block shrinks from 19×19 to 13×13 standalone. Under the operator split the integrated width is **15**, not 9: with the mobile species frozen, `acc_mode=2` (`ACC_STATE_RELAX`, `N_RLX_FROZEN=15`) is what `red_dim()` returns. The narrower 9×9 form is `acc_mode=1`, which is not used — it measured *slower*.

Cost. Modest at this size, and honestly so: at $n = 19$ dense linear algebra is not where the time goes.

Change 38. One SUNDIALS workspace per OpenMP thread

Solver objects are constructed once per thread and reused across cases instead of being constructed and destroyed per case.

Why. Profiling with the integration window collapsed to zero showed per-case construction and destruction of SUNDIALS objects consuming 1.31 s of thread time against 0.2 s of actual solving in a 256-point coupling substep. The measurement redirected the entire effort.

Code. A thread-local workspace pool in the batch driver.

Cost. $1.67\times$ **on end-to-end march wall time** at $q = 1024$ quadrature points — more than the two changes above combined. This is the clearest case in the project of measurement beating expectation: the two changes that were requested for speed delivered accuracy, and the one that delivered speed was not on the list until the profile was read.

8 The fast solve

The fast solve is the tightly coupled part of the split and therefore where essentially all the wall time is: on the 500 nm reference march, 1 h 05 m of fast solves against 12 m of slow substeps. It was instrumented before it was touched, and `compute()` — the preconditioner factorization — was found to hold 85–87%.

Change 39. Skip the elastic solve when it can only return zero

The fast step now sets `useElasticDeformation=0` and `useElasticDeformationFEM=0` when the staged case has no applied load and no dislocations.

Why. `useElasticDeformation=1` costs ~ 42 s per step on a 72 345-DOF solve for a case with no dislocations and no applied stress — roughly 20 minutes of a 30-step run, spent computing a field that is identically zero.

Code. `staging.inputs.elastic_is_trivial(sim_dir)` returns `(trivial, reason)`, and `qssa.MobileQSSASolver` takes an `elastic=` argument. The *reason* is returned rather than a bare boolean so that a run which unexpectedly keeps the elastic solve says why.

Cost. $1.86\times$ on the 500 nm case, bit-identical.

Change 40. Remove AcIR, a matrix assembled every iteration and never read

Why. Dead work in the innermost loop.

Code. One deletion in `ClusterDynamicsFEM.cpp`.

Cost. 1.64× — the single largest of the four fast-solve changes, and it removed code rather than adding any.

Change 41. Cache the Dirichlet selection matrix; build A_1 by filtering

The selection matrix T and its index map are cached, and $A_1 = T^T A T$ is replaced by a filtered copy of A 's nonzeros.

Why. T is a *selection* matrix, so $T^T A T$ is a submatrix — forming it as two sparse products is doing general-purpose work to accomplish a filter.

Code. `rMap` added to `FixedDirichletSolver.h`, along with a new `computeFromTriplets<BilinearWeakFormType>()`.

Cost. ~1.00× **alone** — it measured as nothing, because `rSolver` was rebuilt every iteration and discarded the cache before it could pay. Reported at its measured value; Change 43 is what made it count.

Change 42. Parallelize the element assembly

`BilinearWeakForm::globalTriplets` parallelized over explicit contiguous chunks, concatenated in chunk order.

Why. Element assembly is embarrassingly parallel and was serial.

Code. Chunks are explicit and contiguous, and concatenated *in chunk order*, specifically to preserve the serial triplet sequence. `setFromTriplets` sums duplicate entries, so a different triplet order re-associates those sums and the result would agree only to rounding — which would have destroyed bit-identity as a check. Keeping the order is what makes the verification possible.

Cost. 1.33×.

Change 43. Hoist the loop-invariant weak forms out of the Newton iteration

`R1c`, `bWF_R1c`, `R1c`, `bWF_R1c`, the triplet head and tail, and `rSolver` were moved above the Newton loop; each iteration splices `head + R2 + tail`.

Why. Only the R_2 block depends on the current iterate. Everything else was being rebuilt from scratch every iteration.

Code. Per-iteration work reduces to assembling one block and splicing it between two cached triplet ranges.

Cost. 1.11× on its own — and it is what finally paid off Change 41, since `rSolver` now survives across iterations.

Cumulative, Changes 40–43: on one isolated `DDomp` call at 200 nm, 96.6 s → 37.8 s, **2.556×**, with the CD block (27 720, 12) **bit-identical**.

Change 44. The optimized fast solve measured on a full march

Two complete 500 nm hexagonal marches were run back to back on an otherwise idle machine, differing *only* in the `DDomp` binary.

Why. An isolated-call speedup is not a march speedup: Amdahl applies, and the slow side is untouched.

Code. `studies/compare_architectures.py` reports the physics first and the timings second, and uses **the slow step as a control** — it runs the same code in both runs, so a ratio far from 1.0 there means the two runs were not measured under comparable load and the fast-side number must not be quoted.

Cost.

quantity	reference	optimized	speedup
march total	1 h 18 m 05 s	42 m 41 s	1.829 ×
fast solves (8× DDomp)	1 h 04 m 53 s	29 m 49 s	2.176 ×
one fast solve	8 m 06 s	3 m 43 s	2.176×
slow substeps (control)	12 m 18 s	11 m 58 s	1.028×

The state array (9,90 617, 19) is **bit-identical** between the two runs, and the control ratio of 1.028 confirms the machine was comparably loaded. The march speedup is below the isolated 2.556× for two reasons, both expected: the slow side is untouched, and the first interval takes no fast solve at all.

Part VII

Portability, reproducibility and post-processing

Change 45. One build script, one CMakeLists, every platform

Why. The tree only built on the platform it was last built on. The build script `sed`-patched `MoDELlib3/CMakeLists.txt` on every run — rewriting the Eigen path, deleting the Qt lines, commenting out `add_subdirectory(DDqt)` — so the source tree carried whichever platform’s edits had been applied last. On macOS the build stopped at the first translation unit.

Code. Four platform-specific failures, none of which Linux ever shows:

Symptom	Cause
<code>clang: error: unsupported option '-fopenmp'</code>	the flag was hard-coded; Apple clang needs <code>-Xpreprocessor -fopenmp</code> and Homebrew’s <code>libomp</code> , which <code>find_package(OpenMP)</code> now supplies
use of undeclared identifier <code>'assert'</code> , 103 files	those files never included <code><cassert></code> ; <code>libstdc++</code> leaks it in, <code>libc++</code> does not. A tree-wide <code>-include cassert</code> substitutes for editing all 103 and keeps the fork mergeable
must explicitly initialize the const member <code>'invTrD'</code>	Eigen $\geq 3.4.90$ declares <code>Matrix() = default</code> , so a const member with no initializer is ill-formed. <code>invTrD</code> is unused; now zero-initialized. An Eigen-version dependency, not a platform one
no viable conversion from <code>'RationalLatticeDirection<3>'</code>	clang resolves the dependent <code>this->operator+(...)</code> call against the <code>LatticeVector</code> overload alone; naming the temporary fixes it

Every dependency is now searched for, every optimization flag probed with `check_cxx_compiler_flag` before use, and everything optional degrades to a status line. `DDqt` is built when Qt6 and VTK ≥ 9.4 are both found and skipped otherwise. `cmake_minimum_required` went from 3.1.0 to 3.16 — CMake ≥ 4.0 refuses to run a project asking for less than 3.5, so the old floor was a time bomb.

Cost. None. `DDomp`, `microstructureGenerator` and `libMoDELlib.a` all build clean on Apple silicon, and the CD Newton iterates reproduce *bit-identically* between two differently configured builds of the same tree.

Change 46. Eigen 3.4.x required, and the version test made fatal

Why. This is the worst kind of portability failure: a clean build and a wrong answer, reported as a solver problem. Homebrew’s `eigen` formula is 5.x, and `MoDELlib` built against it *compiles* — then the BiCGSTAB solve inside the mobile Newton iteration breaks down on the first step (`Iterative`

FixedDirichletSolver failed) on a 500 nm case that had staged successfully on Linux/Eigen 3.4.0 two months earlier. Rebuilding the same source against 3.4.0 and re-running the same case bootstraps in 138 s.

Code. `build_modelib.sh` fetches 3.4.0 into `Libraries/eigen-3.4.0` when the system has only Eigen 5; `CMakeLists.txt` prefers that copy; the version check is *fatal*, with `-DEIGEN3_ALLOW_UNTESTED=ON` to override.

The version test has to read *both* places Eigen states its version: 3.4.x puts the macros in `Eigen/src/Core/util/Macros.h`, 5.x in `Eigen/Version`, and 5.0.1 spells itself 3.5.0 there (`EIGEN_WORLD_VERSION` stays 3 forever) — so a naive `~3\.` test passes it on both counts.

Cost. None.

Change 47. No build tree in git; preflight runs the binaries

Why. `MoDELib3/build_dc/` — `libMoDELib.so`, `DDomp`, and a `build.ninja` carrying 374 absolute `/mnt/d/...` paths — used to be committed. A clone on another machine got that Linux ELF, `paths.modelib_ddomp()` reported `MoDELib` as built, `ensure_modelib` skipped the build, and the first `DDomp` call failed.

Code. Both build trees are ignored, and `build.preflight()` *runs* each binary rather than trusting its presence.

A related silent failure: moving or renaming the repository invalidates both build trees, because a CMake cache stores absolute paths. The failure is quiet — **the already-built binary keeps working**, so nothing breaks until the next reconfigure, possibly months later. `build.stale_cache_home()` reports it, `build.describe()` flags it, and `ensure_zrmicro_solver()` discards and rebuilds. That check runs *before* the “already built” shortcut — putting it after, as an earlier revision did, made it unreachable in exactly the situation it exists for.

Cost. Seconds per run, against hours of a wrong one.

Change 48. `ev1/cdNodes.txt` — the CD node coordinates

Why. Without it the fields cannot be placed in space: the CD trial functions live on second-order elements (24115 nodes for a 3392-node mesh), so the field rows do *not* correspond to the `.msh` vertices and the ordering is not reproducible externally.

Code. Written once per run.

Cost. Negligible.

Change 49. Geometry-true field rendering

Why. Nothing in the figure code should be specific to one domain shape, and several things were.

Code. Every shape decision — the outline, the hidden-line test, the volume that turns a density into a count, the region loops are placed in, the clip — comes from `fields.domain_faces / domain_edges / domain_volume`, the convex hull of the CD node cloud. A cubic case gives 6 faces, 12 edges and `domain_volume == prod(hi - lo)` to machine precision, so the box case is untouched; hexagonal gives 8 faces and 18 edges.

Three specific corrections, each of which had been silently wrong on a non-cubic domain:

- *Both cut planes go into ONE Poly3DCollection.* Matplotlib depth-sorts 3-D *artists* by a single scalar each, so two intersecting surfaces drawn as separate `plot_surface` calls cannot interleave and whichever sorts in front hides the other *completely* — which is what used to happen, the vertical cut covering the horizontal one entirely.
- *The panel zoom is fitted, not fixed.* A 3-D axes clips its artists at the axes rectangle, not at the data cube, so a zoom that projects the domain taller than the panel silently cuts the ends off the wireframe. It is the *aspect ratio* that decides this, not the size: a cube fits at 1.22 and is left exactly as it was, while a 1 : 0.87 : 1.6 hexagonal prism projects to `-0.068..1.054` and loses both end faces — identically at 200 nm and at 500 nm. `_fit_zoom` reduces the zoom until the convex *hull* fits, measured through matplotlib’s own `get_proj`. Fit to the hull, not the bounding box, whose corners for a prism sit out in the six empty wedges.

- *The volume that turns a density into a count* is `domain_volume`, not Π (bounding box) — for a hexagonal prism the box is 4/3 the crystal, so the discrete loop count used to come out a third too high.

Cost. The loop-overlay panels are the slow figures at ~ 16 s each; the rest are seconds.

Status. `volume_average.averaged_trajectory` still calls `voronoi_weights` *without faces*, so on a non-box domain its volume weights carry the same bounding-box error. That is deliberate — fixing it changes already-published figures — but it is wrong for a hexagonal run. See Part VIII, item 4.

Change 50. Interpolated movie frames

`OUTPUT['movie_interp']` (default 5, 1 disables) subdivides each dose interval into that many frames.

Why. The march writes *one* snapshot per dose interval, so an eight-dose run animates as an eight-frame flipbook however high the fps.

Code. The extra frames are interpolated, not solved, and every synthetic frame is labelled (`interp`) in its title. Two things about the blend are load-bearing:

- It is *geometric* wherever both endpoints are positive (linear otherwise — the pristine seed sits at dose 0). The fields grow by decades through the nucleation transient, where a linear interpolant would sit at the upper endpoint for the whole interval, and geometric blending commutes with the power-law reductions the figures draw: a size $d \sim (c/N)^{1/3}$ formed from blended c and N is the blended d to 2×10^{-15} .
- It is taken on the *CD block*, not on the 19-column state it is built from. `immobile_0d_to_modelib` sums species, and a sum of geometric blends is not the geometric blend of the sums, so blending the state first yields frames that are *not bracketed by their neighbors* — visible as a population overshooting and falling back.

`cd_blocks` deliberately keeps `interp=1` as its default: `discrete_loops` and the `3d//gb/` panels must see solved states only. Only `movies.render` opts in.

Cost. Interpolation is free relative to solving; it is what makes a smooth movie possible without more fast solves.

Change 51. Computational statistics in the run provenance

`march_report.computational_stats` writes wall clock total and per solve, the number of ODEs actually integrated, node counts and per-node rates into `report.md`.

Why. Cost claims about this architecture were being made from memory. They should come from the run.

Code. Reports the *integrated* ODE count, not the carried state length — 15 per node under the operator split, not 19.

Cost. Negligible.

Part VIII

Open items

These are unresolved as of 15 August 2026. Two of them are physics decisions rather than defects.

1. **No continuum $\rightarrow$ discrete loop transition.** The sibling `fullCD` branch has `clusterDiscretizationTime` and `initializeDiscreteClimbLoops()`, which convert the continuum loop field into discrete climb loops once the dose passes a threshold. This branch has no such transition. It is not a regression — nothing of the kind existed in `MoDELib2-NNL` to lose — but it is the largest capability the two branches do not share, and it is the one

worth taking from fullCD. The post-processing `discrete_loops` module constructs a discrete population from the continuum fields for *visualization and export*, which is not the same thing — it does not feed back into the solve.

2. **The mobile Newton iteration does not converge from a cold start.** `convergenceError` = 2.9×10^5 after the first iteration, then the BiCGSTAB solve fails on the second. The cause is the cold start: `mobileClusters` is initialized to *thermal equilibrium* ($\sim 10^{-15}$) and must reach the irradiation steady state ($\sim 10^{-6}$) — nine orders — in one Newton solve of a quadratically nonlinear system. Switching `rSolver` to the direct SparseLU branch was tried and rejected: it fails *earlier*, in `compute()` itself, on the $\sim 96\text{k}$ -DOF matrix. Note that dt does not enter the mobile equation at all, so ramping the dose step cannot help; the remedy has to be a better initial guess — which is exactly what the two-time-scale coupling provides — or continuation in G . Change 20 removed a factor of 3.98 on the diffusion operator after this was diagnosed, so it should be re-tested.
3. **C_{2i}/C_{3i} depleted $\sim 2.5\times$ once the loops have grown.** At 1 dpa they agree with the 0-D to 1–2%; by 5 dpa they have fallen to 0.41–0.45 $\times$ and stay there. ZrMicro’s loops absorb $2i$ and $3i$ for growth — $\text{flux}_i = \omega_i(C_i + 2C_{2i} + 3C_{3i})$ — but its dC_{2i}/dt and dC_{3i}/dt never debit those pools. Zr3d_ghoniem’s `ImmobileSinks` *does* debit them, so the 3-D carries a $2i/3i$ loop sink the 0-D lacks. The one-line estimate $C_{2i}^{3D}/C_{2i}^{0D} = \Lambda_{0D}/(\Lambda_{0D} + D_i k_{\text{loops}}^2)$ gives $20.92/(20.92 + 17.86) = 0.54$ against the 0.41 measured — same mechanism, right order.

Not fixed, because it is a physics decision, not a bug in either code. Zr3d_ghoniem is the mass-conserving one: a loop that absorbs a di-interstitial must remove it from the mobile pool. The reconciling change belongs in ZrMicro, but it would shift the 0-D fit that the experiments are calibrated against.

This matters beyond itself: the clustering nucleation flux of Change 4 is $K C_i C_{3i}$, so a depleted C_{3i} feeds it directly. The 3-D’s own nucleation rate is $0.782\times$ the 0-D’s for this reason, and $0.704/0.782 = 0.90$ — i.e. with the cluster concentrations matched, the $\langle a \rangle$ density would land within 10%. **Closing this should close most of what remains on the $\langle a \rangle$ families as well.**

4. **The boundary-flux ratio sits at 0.63–0.65 on the 500 nm march**, against 0.963 at 10 dpa on the 200 nm case (Change 35). The leading hypothesis, untested, is the bounding-box Voronoi weights in `averaged_trajectory` noted under Change 49: they over-weight the boundary nodes, and on a hexagonal prism the bounding box is $4/3$ the crystal.
5. **The 28-parameter fit is stale** after the atomic-volume correction of Change 31. The loop densities halve. A refit is planned separately and is deliberately *not* attempted here.
6. **DAD is a fitting form, not a generated one.** `dadAnisotropy/dadZ0` are fitted to reproduce the 0-D symmetric bias forms rather than derived from D_c/D_a , and the diffusion tensors are isotropic to match ZrMicro. These runs therefore exercise none of the tensorial-DAD mechanism of the deliverable, and the fit is tied to 573 K.
7. **SIPN stress weights $w_k(\sigma)$ are not wired.** Exact at zero applied stress, which is the configured condition for every run reported here.
8. **Warning: Sum of R2 is not zero** remains, and is now *meaningful* rather than merely benign: the reaction map contains $i+3i$ but there is no $4i$ species to receive the product. Those interstitials are no longer unaccounted — they are exactly the loop embryos of Change 4, and the immobile solve now credits them. ZrMicro truncates the cluster ladder at the same place for the same reason.
9. **Near-boundary loop densities are not quantitative.** Cascade nucleation is spatially uniform, but the only loop-removal channel is coalescence, driven by the absorbed mobile flux

— which vanishes where Dirichlet pins the mobile concentrations. The boundary is a sink for mobile defects but **not** for loops. Of the 24 115 CD nodes on the 1 μm cube, 6522 (27%) lie on the six Dirichlet faces, and at 21 dpa they carry **99.8%** of the total $\langle a \rangle$ density. **Never report a whole-domain mean of a loop quantity on such a geometry** — quote the interior mean, the innermost quartile by distance to the nearest face. The difference is not cosmetic: at 21 dpa the coupled march’s $\langle a \rangle$ density is $474\times$ the 0-D as a domain mean and $0.75\times$ it in the interior.

That the coupled march reproduces this artifact is a point *in favor of* the coupling: the same boundary behavior emerges whether MoDELib3’s nodal scheme or ZrMicro’s CVODE march advances the immobile population.

Verification status against the 0-D

Ratios 3-D / 0-D at interior nodes, from the 31-step run to 30 dpa. “Before” is cascade-only nucleation, “after” is with the homogeneous SIA clustering flux of Change 4.

quantity	1 dpa	5 dpa	10 dpa	30 dpa
C_v	1.073	0.982	0.994	1.004
C_i	1.145	1.112	1.133	1.148
C_{2i}	1.012	0.412	0.394	0.388
C_{3i}	0.977	0.400	0.383	0.377
$\langle c \rangle$ loop density	0.999	0.999	0.999	0.999
$\langle a \rangle$ loop density — before	0.426	0.547	0.540	0.533
$\langle a \rangle$ loop density — after	0.662	0.739	0.718	0.704
$\langle a \rangle$ loop content — before	0.486	0.634	0.627	0.620
$\langle a \rangle$ loop content — after	0.639	0.811	0.795	0.784
$\langle a \rangle$ defects/loop — before	1.141	1.159	1.159	1.164
$\langle a \rangle$ defects/loop — after	0.965	1.098	1.106	1.114

Interior nodes are selected by the top decile of C_v , and **that selection must be made ONCE from the mobile field and applied to all quantities**. Loop density anti-correlates with C_v ($\text{corr} = -0.98$), because near the boundary the mobile species are depleted so loops neither grow nor coalesce and simply accumulate at their nucleation value. A per-quantity percentile would pick the boundary for the immobile fields and the interior for the mobile ones.

C_v agrees to within 2% beyond 5 dpa and the $\langle c \rangle$ loop density to 0.1% at every dose. C_i runs 11–15% high throughout, at every dose and in both runs. The two remaining discrepancies are one story, not two — see Part VIII, item 3.

Note on scope. The table above predates the atomic-volume correction of Change 31. Both codes changed together and by the same factor, so the *ratios* are expected to survive it, but they have not been re-measured. They will be, as part of the planned refit.

Summary of measured efficiency changes

Change	What	Measured
2	nodal immobile update, no mass-matrix projection	avoids a CG solve per dose step
8	vacancy-loop dissolution implicit	unconditional stability (was $\times 35$ over)
12	coalescence implicit	unconditional stability (was $\times 3.6$ over)
21	clamp deferred out of the Newton loop	$3.5\times$, and quadratic convergence
27	operator split, mobile frozen	1.0×10^{10} in factorization flops
36	AD Jacobian	accuracy, not wall time
37	accumulators as quadratures	$19 \rightarrow 13$ standalone; 15 split
38	one SUNDIALS workspace per thread	1.67 $\times$ end to end
39	skip the trivial elastic solve	1.86 $\times$, bit-identical
40	remove dead AcIR assembly	$1.64\times$
41	cached T , filtered A_1	$\sim 1.00\times$ alone
42	parallel element assembly	$1.33\times$
43	hoist invariant assembly	$1.11\times$
44	40–43 on a full march	2.176 $\times$ fast, 1.829 $\times$ march

The fast solve remains the bottleneck, and `setFromTriplets` is now its largest single component. The report step is $\sim 47\%$ of a full run’s wall time, which is the next thing worth attention.